UNIVERSITY OF CAMBRIDGE

MPhil in Machine Learning and Machine Intelligence 2025-2026

Module Coursework Coversheet

Blind Grading Number: 8864G
Module Code: 4F13
Module Name: Probabilistic Machine Learning
Coursework Number or Name: 1: Gaussian Processes
Word or Page Count (if limit prescribed): 5 pages (cover sheet excluded)

Declaration

☑ *tick to confirm*

I hereby declare that this piece of work is my own unaided effort and adheres to the Department of Engineering's guidelines on plagiarism.

This piece of work is to be marked according to the following grading scheme:

Distinction		**Pass**				**Fail (C+ - marginal fail)**		
Outstanding	**A+**	**A**	**A-**	**B+**	**B**	**C+**	**C**	**Unsatisfactory**
90-100	80-89	75-79	70-74	65-69	60-64	55-59	50-54	0-49
Late Penalty: 10% of mark for each day, or part day, late (Sunday excluded).								

Coursework 1: Gaussian Processes

Question A

Figure 1 shows the result of training a Gaussian Process (GP) using a Squared Exponential (SE) covariance function. The model initial hyperparameters are in ***Table 1***, and subsequently optimized by minimizing the negative log marginal likelihood (NLL), which is given as follows.

$$NLL = \frac{1}{2}y^T(K + {\sigma_n}^2 I)^{-1}y + \frac{1}{2}log|K + {\sigma_n}^2 I| + \frac{n}{2}log(2\pi)$$

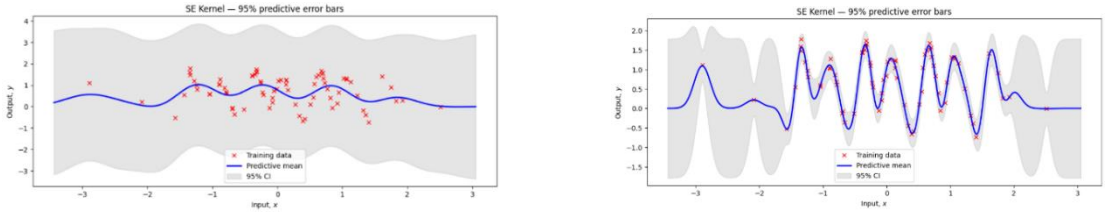
It is the combination of data fit term, complexity penalty and a constant. The key code to achieve this is shown in ***Command 1***.

Command 1 *Train a GP with a squared exponential covariance function*

```
k = GPy.kern.RBF(input_dim=X.shape[1], lengthscale=np.exp(-1), variance=1.0)
m = GPy.models.GPRegression(X, y, k)
m.likelihood.variance = 1.0
m.optimize(optimizer='lbfgsb', max_iters=2000, messages=False)
```

Table 1 *Model Hyperparameters & Negative Log Marginal Likelihood*

	length scale (ℓ)	signal std (σ_f)	noise std (σ_n)	Negative Log Marginal Likelihood
Initial Hyperparameters	e^{-1}	1.000	1.000	92.910
Optimised Hyperparameters	0.128	0.896	0.117	11.899



(a) Before optimization *(b) After optimization*

Figure 1 *Gaussian Prediction using SE function*

We can see a significant drop of NLL after optimization from 92.91 to 11.9. The length-scale (l) was drastically reduced to 0.128, allowing the model to capture the data's high-frequency oscillations by making correlations decay rapidly with distance. Because this more flexible function could now closely track the data, the optimizer concluded the observations were not noisy, thus significantly reducing the noise variance $({\sigma_n}^2)$ and consequently increasing its reliance on the data. The signal variance $({\sigma_f}^2)$ also decreased slightly, adjusting the model's prior output amplitude.

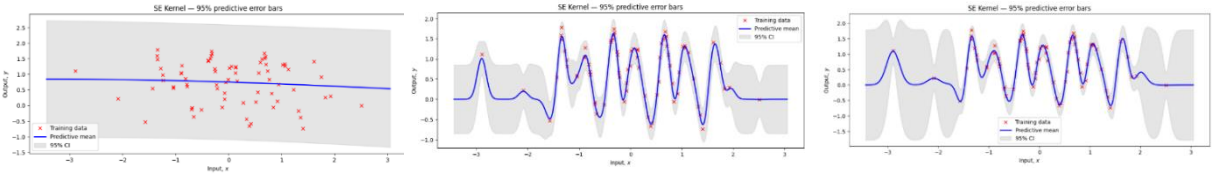
These optimized parameters directly dictate the shape of the 95% confidence intervals, which are based on the predictive variance $\sigma^2(y_*)$. This variance is mathematically defined as the prior variance ${\sigma_f}^2$ minus a reduction term ${k_*}^T(K + {\sigma_n}^2 I)^{-1}k_*$, which depends on the test point's (x_*) correlation k_* to the training data. In data-dense regions, x_* is close to the data, k_* is large, and the

variance collapses to the minimal noise variance, resulting in thin error-bars. In data-sparse regions, x_* is far from the data. The small l ensures k_* approaches zero, nullifying the reduction term. The variance thus reverts to its maximum prior value ($\sigma_f{}^2$), causing the error-bars to widen significantly. We can visually see this behavior in ***Figure 1***.

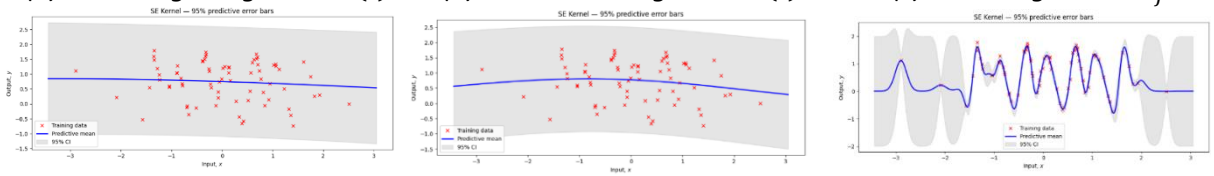
Question B

Table 2 *Models' Hyperparameters & Negative Log Marginal Likelihood*

	Initial Hyperparameters			Optimised Hyperparameters			Negative Log Marginal Likelihood
	length scale (ℓ)	signal std (σ_f)	noise std (σ_n)	length scale (ℓ)	signal std (σ_f)	noise std (σ_n)	
Model a	8.000	1.000	1.000	8.042	0.696	0.663	78.220
Model b	0.002	1.000	1.000	0.089	0.397	0.124	36.829
Model c	e^{-1}	10.000	1.000	0.128	0.897	0.118	11.899
Model d	e^{-1}	0.001	1.000	3.040	0.449	0.624	78.857
Model e	e^{-1}	1.000	10.000	8.042	0.696	0.663	78.220
Model f	e^{-1}	1.000	0.001	0.115	1.103	0.050	62.063



(a) With large length scale (l) *(b) With small length scale (l)* *(c) With large initial σ_f*



(d) With small initial σ_f *(e) With large initial σ_n* *(f) With small initial σ_n*

Figure 2 *The effect of different initial hyperparameters*

The optimization, when started from different random initializations, consistently converges to a few distinct local optima, each representing a different interpretation of the data, as we can see clearly in ***Table 2*** and ***Figure 2***. Models a, d and e converge to an underfitting smooth solution, ignoring the data or mistaking the signal for high noise. Model b finds an overfitting wiggly solution from a tiny initial l. Model f finds an overconfident noise-free solution by starting with near-zero noise, resulting in an overly flexible function with unrealistically small error-bars. But we can see model c finds the best fit, consistent with the result in question A.

The NLL serves as a principled metric for model selection. Model c is unequivocally the most plausible model, as its NLL of 11.899 is dramatically lower than the next-best alternative, $\Delta NLL = (36.829) - (11.899) \approx 25$. This gap is the log bayes factor, which quantifies the evidence for one model over another. A log Bayes factor greater than 5 (and certainly > 10) is conventionally considered decisive or overwhelming statistical evidence. Our value means the data is e^{25} times more likely under the best model than the next-best one, providing extreme confidence in this conclusion.

Question C

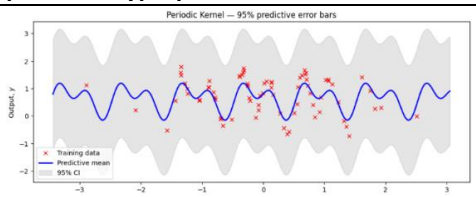
A GP with a periodic covariance function was trained on the same data as used in previous questions, as shown below.

Command 2 *Train a GP with a periodic covariance function (same as* ***Command 1*** *except this line)*

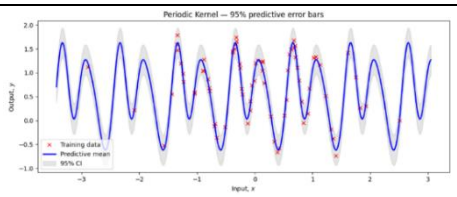
```
k = GPy.kern.StdPeriodic(input_dim=1, variance=1.0, lengthscale=1.0, period=1.0)
```

Table 3 *Periodic Model Hyperparameters & Negative Log Marginal Likelihood*

	length scale (ℓ)	signal std (σ_f)	noise std (σ_n)	period (p)	Negative Log Marginal Likelihood
Initial Hyperparameters	1.000	1.000	1.000	1.000	83.407
Optimised Hyperparameters	0.515	1.164	0.109	0.998	-35.266



(a) Before optimization



(b) After optimization

Figure 3 *Gaussian Prediction using periodic covariance function*

Through ***Table 3*** we can see $NLL = -35.266$, which is decisively superior to the optimal SE kernel's NLL of 11.899. This confirms the periodic hypothesis is far more plausible.

The predictive **error-bars** highlight their core difference. The SE kernel's covariance depends only on distance, $k(x, x') = f(|x - x'|)$. As a test point x_* moves far from the training data, its correlation to all data points decays to zero, causing the predictive variance to revert to the high prior variance. In contrast, the Periodic kernel's covariance depends on the phase as $k(x, x') = f\left(sin^2\left(\frac{\pi|x-x'|}{p}\right)\right)$, which makes points x and $x + p$ highly correlated. Therefore, its predictive uncertainty is also periodic, it grows in data-sparse phases but shrinks again as x_* aligns with the data-dense phases of previous cycles, correctly modeling a repeating structure.

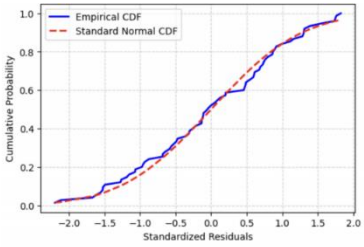


Figure 4 *Empirical vs Theoretical CDF of Residuals*

To verify that the data-generating mechanism is strictly periodic plus noise, we test if the model's standardized residuals $z = \frac{y_{real} - y_{pred}}{\sigma_{pred}}$ follow a standard normal $\mathcal{N}(0, 1)$ distribution. ***Figure 4*** shows the empirical CDF of the residuals almost perfectly matching the theoretical normal CDF. A Kolmogorov-Smirnov (K-S) test confirms this visually, yielding a $p - value \gg 0.05$. Since we cannot

reject this null hypothesis, this provides strong evidence that the data is generated by a periodic function corrupted by Gaussian noise. But we can see in ***Figure 3(b)*** that the data has a general negative trend which isn't periodic. A strictly periodic function cannot capture this. Therefore, the data is likely quasi-periodic, perhaps a combination of periodic and linear terms.

Question D

We sample $f \sim \mathcal{N}(0, K)$ using the Cholesky decomposition. The covariance matrix K is defined by the product of an RBF kernel and a Periodic kernel. The central commands to achieve this are as follows.

Command 3 *Generate data from GP*

```
k_rbf = GPy.kern.RBF(input_dim=1, lengthscale=np.exp(2), variance=1.0)
k_per = GPy.kern.StdPeriodic(input_dim=1, lengthscale=np.exp(-0.5), period=1.0, variance=1.0)
kernel_prod = k_rbf * k_per
K = kernel_prod.K(X_sample)
K += jitter * np.eye(N)
L = np.linalg.cholesky(K)
```

Sampling from the GP prior requires a Cholesky decomposition $LL^T = K$. This algorithm strictly demands a positive definite matrix. A small diagonal $jitter$ is added to K to provide numerical stability by correcting for floating-point errors that can make the matrix non-positive definite, thus guaranteeing the decomposition's success.

(a) Samples from -5 to 5 *(b) Samples from -20 to 20*

Figure 5 *Generating Sample functions with random seeds*

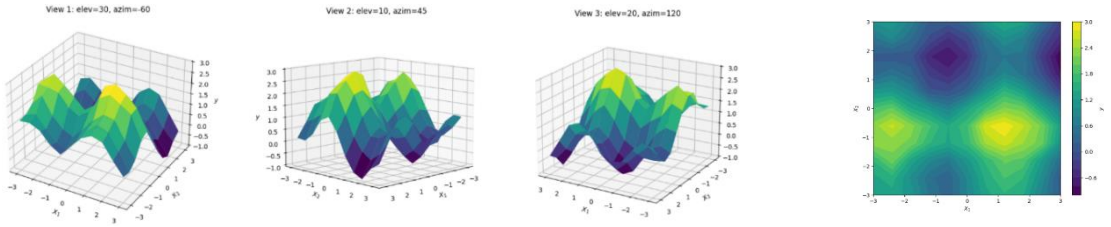
The properties of the sampled functions in ***Figure 5*** are directly dictated by the form of the composite covariance function, and the kernel function is as follow.

$$k(x, x') = {\sigma_{f_1}}^2 \exp\left(-\frac{(x-x')^2}{2{l_1}^2}\right) \times {\sigma_{f_2}}^2 \exp\left(-\frac{2}{{l_2}^2} sin^2\left(\frac{\pi(x-x')}{p}\right)\right)$$

The periodic kernel's covariance form enforces the fast, repeating oscillations by making points separated by one period highly correlated. The RBF kernel's covariance, set with a large length-scale, creates a smooth, slowly-varying envelope. This results in functions that are locally periodic but whose global amplitude is modulated by the RBF. ***Figure 5(a)*** shows the fast oscillations, while the wider view in ***Figure 5(b)*** reveals the long-range RBF envelope changing the overall amplitude.

Question E

The visual complexity shown in ***Figure 6*** suggests that a simple model with a single smoothness assumption might be unable to capture the data, and a more complex model might be required.

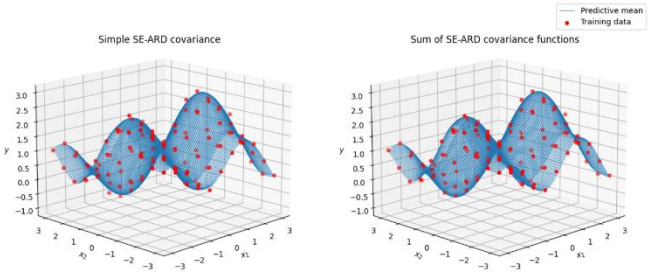


(a) 3D visualization *(b) up-down visualization*

Figure 6 *Visualize cw1e.mat dataset*

Table 4 *Hyperparameters of single and double kernel models*

	Optimised Hyperparameters									
	length scale (SE, ARD)		**length scale'** (SE', ARD')		**signal std** (σ_f)	**signal std'** (σ_f')	**noise std** (σ_n)	**LML**	**Data fit term**	**Complexity penalty**
	λ_1	λ_2	λ_1'	λ_2'						
Model 1 (single)	1.512	1.286	—	—	1.107	—	0.103	19.219	-60.501	-190.910
Model 2 (double)	1.447	136121.063	98444.856	0.986	1.110	0.710	0.098	66.408	-60.500	-238.100



(a) Single kernel *(b) Double kernel*

Command 4 *Single & double kernel models*

```
# single
k1 = GPy.kern.RBF(input_dim=2, ARD=True, lenghscale=l1, variance=v1)
m1 = GPy.models.GPRegression(X, y, k1)
m1.likelihood.variance=1.0

# double
k2a = GPy.kern.RBF(input_dim=2, ARD=True, lenghscale=l2a, variance=v2a)
k2b = GPy.kern.RBF(input_dim=2, ARD=True, lenghscale=l2b, variance=v2b)
k2 = k2a + k2b
m2 = GPy.models.GPRegression(X, y, k2)
m2.likelihood.variance=1.0
```

Figure 7 *Single & Double Kernels Models Comparison &* ***Command 4*** *Single & double kernel models*

For single kernel model, we have the following equation, while double one uses a sum of that:

$$k_1(x, x') = {\sigma_f}^2 \exp\left(-\frac{1}{2}\sum_{d=1}^{2}\left(\frac{x_d - {x_d}'}{\lambda_d}\right)^2\right)$$

Since the exponent of the ARD function is given as the sum of the effective squared distances, large distance in only one dimension will let $k_1(x, x')$ close to zero, making it not flexible enough to detect the correlation along each axis separately and unable to capture the ridge-like structures in data. In contrast, model 2 found a highly flexible, anisotropic solution by effectively decomposing the data into two 1D functions, one kernel captured the variation along the x_1 axis while the other for x_2.

The Log Marginal Likelihood (**LML**) shows that Model 2 is decisively better (LML = 66.4) than Model 1 (LML = 19.2). ***Table 4*** shows both models achieved an identical **Data Fit** score, but model 2 has low **Complexity penalty**, proving that while both observe points equally well, model 2's additive, anisotropic structure was a vastly more plausible statistical explanation for the data, resulting in a more confident model and a much higher LML.

This explains why the marginal likelihood is much higher for the two kernels model. In conclusion, the second model is decisively superior to the first model.